VIT – School of Computer Science Engineering and Information Systems

BCSE302L – Database Systems | Course Project

REVIEW I – Problem Analysis & Database Design

Project Title: Smart Hospital Management System with Emergency Triage, Bed Allocation and Audit Logging

Team Members

S.No.	Name	Register Number
1	Anushka Jaiswal	24BYB0121
2	Shubham Mishra	24BYB0117
3	Aalap Panda	24BYB0191

1. Problem Definition & Scope

1.1 Problem Statement

In many hospitals, patient details, doctor appointments, prescriptions and bills are kept in paper files or in separate Excel sheets. Because the data is scattered, the same patient gets registered again and again, two patients can be booked with the same doctor at the same time, prescription records get lost and bills are calculated by hand with mistakes.

The problem is bigger in the emergency department, where many patients arrive together. They are often attended to in the order they came instead of how serious their condition is, so a critical patient can end up waiting behind someone with a minor injury. Bed availability is also checked manually, so a bed shown as free may already be taken. Patient records are also sensitive, but most systems keep no proper record of who changed what and when.

This project proposes a relational database application that keeps all this data in one place with proper keys and constraints. A patient is registered once and can visit through a normal appointment or as an emergency case. Emergency patients get a triage (priority) level so critical cases are treated first. The doctor treats the patient and writes a prescription, a bed is allotted if admission is needed, and the bill is generated from the consultation fee, medicines and bed charges. Every important change is recorded automatically in an audit log. The business operations are implemented using SQL and PL/SQL (procedures, functions, cursor and triggers), and the application is connected to the database through a programming language.

1.2 Objectives

1. To design a normalised relational database of 8 tables for the hospital with proper primary keys and foreign keys.
2. To handle normal appointments and emergency visits in the same database.
3. To avoid duplicate patient records and clashing doctor appointments using constraints and checks.
4. To prioritise emergency patients using a triage level.
5. To record prescriptions and reduce medicine stock automatically using triggers.
6. To track bed status (Free, Occupied, Cleaning) and avoid allotting a bed twice.

7. To generate the bill automatically using functions and procedures.
8. To maintain an automatic audit log of inserts, updates and deletes on important tables.
9. To retrieve useful information through SQL queries (joins, aggregation, subqueries) and reports.
10. To demonstrate the working of the database through an application connected to it.

1.3 Users of the System

User	Role in the system
Admin	Manages doctors, medicines, beds and reports; views the audit log
Receptionist / Triage Nurse	Registers patients, books appointments, records emergency cases and assigns the triage level
Doctor	Views assigned visits and writes prescriptions
Billing Staff	Generates bills and records payments
Patient	Receives treatment and prescription, and pays the bill

1.4 Scope

Included

- Patient registration and visit history
- Doctor management with department and availability
- Appointment booking and cancellation
- Emergency case recording with triage level
- Prescription and medicine stock management
- Bed management, admission and discharge
- Billing and payment records
- Automatic audit logging
- Reports using SQL queries
- Database connectivity with an application (Python / Java)

Not included

- Laboratory and radiology management
- Online payment gateway and insurance claims
- Ambulance tracking
- Staff salary and attendance
- Multi-branch hospital network

Assumptions

- Every visit is handled by one main doctor and has one bill.
- One visit uses at most one bed (only if the patient is admitted).
- Department is stored as an attribute of the doctor to stay within the 6–8 table limit of the guidelines.

2. Requirement Analysis

2.1 Major Entities, Attributes and Constraints

The system has 8 entities (tables). For each one, the attributes, data types and constraints are given. PK = Primary Key, FK = Foreign Key.

1. PATIENT

Attribute	Data type	Key / Constraint
patient_id	NUMBER	PK
patient_name	VARCHAR2(50)	NOT NULL
gender	VARCHAR2(10)	CHECK (Male / Female / Other)
date_of_birth	DATE	NOT NULL
phone	VARCHAR2(15)	UNIQUE, NOT NULL
address	VARCHAR2(100)	-
blood_group	VARCHAR2(5)	-
emergency_contact	VARCHAR2(15)	-
registration_date	DATE	DEFAULT SYSDATE

2. DOCTOR

Attribute	Data type	Key / Constraint
doctor_id	NUMBER	PK
doctor_name	VARCHAR2(50)	NOT NULL
department	VARCHAR2(30)	NOT NULL
specialization	VARCHAR2(40)	-
phone	VARCHAR2(15)	UNIQUE
consultation_fee	NUMBER(8,2)	CHECK (consultation_fee > 0)
shift	VARCHAR2(10)	CHECK (Morning / Evening / Night)
availability_status	VARCHAR2(10)	DEFAULT 'Available', CHECK (Available / Busy)

3. BED

Attribute	Data type	Key / Constraint
bed_id	NUMBER	PK
ward_type	VARCHAR2(15)	CHECK (General / Private / ICU / Emergency)
bed_status	VARCHAR2(10)	DEFAULT 'Free', CHECK (Free / Occupied / Cleaning)
daily_charge	NUMBER(8,2)	CHECK (daily_charge > 0)

4. VISIT (appointments and emergency cases)

Attribute	Data type	Key / Constraint
visit_id	NUMBER	PK
patient_id	NUMBER	FK → PATIENT, NOT NULL
doctor_id	NUMBER	FK → DOCTOR, NOT NULL
bed_id	NUMBER	FK → BED, can be NULL (only for admitted patients)
visit_type	VARCHAR2(12)	NOT NULL, CHECK (Appointment / Emergency)
visit_datetime	TIMESTAMP	NOT NULL
complaint	VARCHAR2(100)	-
triage_level	NUMBER(1)	CHECK (1 to 4), NULL for normal appointments
visit_status	VARCHAR2(15)	DEFAULT 'Scheduled', CHECK (Scheduled / Waiting / Under Treatment / Admitted / Discharged / Cancelled)
admit_time	TIMESTAMP	-
discharge_time	TIMESTAMP	CHECK (discharge_time >= admit_time)

5. MEDICINE

Attribute	Data type	Key / Constraint
medicine_id	NUMBER	PK
medicine_name	VARCHAR2(50)	NOT NULL
manufacturer	VARCHAR2(50)	-
price_per_unit	NUMBER(8,2)	CHECK (price_per_unit > 0)
stock_quantity	NUMBER	DEFAULT 0, CHECK (stock_quantity >= 0)

6. PRESCRIPTION (links VISIT and MEDICINE)

Attribute	Data type	Key / Constraint
prescription_id	NUMBER	PK
visit_id	NUMBER	FK → VISIT, NOT NULL
medicine_id	NUMBER	FK → MEDICINE, NOT NULL
dosage	VARCHAR2(30)	-
duration_days	NUMBER	CHECK (duration_days > 0)
quantity	NUMBER	CHECK (quantity > 0)

7. BILL

Attribute	Data type	Key / Constraint
bill_id	NUMBER	PK
visit_id	NUMBER	FK → VISIT, UNIQUE, NOT NULL (one bill per visit)
consultation_charge	NUMBER(8,2)	DEFAULT 0
medicine_charge	NUMBER(8,2)	DEFAULT 0
bed_charge	NUMBER(8,2)	DEFAULT 0

Attribute	Data type	Key / Constraint
total_amount	NUMBER(10,2)	-
payment_mode	VARCHAR2(10)	CHECK (Cash / Card / UPI)
payment_status	VARCHAR2(10)	DEFAULT 'Pending', CHECK (Paid / Pending)
bill_date	DATE	DEFAULT SYSDATE

8. AUDIT_LOG

Attribute	Data type	Key / Constraint
log_id	NUMBER	PK
table_name	VARCHAR2(30)	NOT NULL
action_type	VARCHAR2(10)	CHECK (INSERT / UPDATE / DELETE)
record_id	NUMBER	-
changed_by	VARCHAR2(30)	DEFAULT USER
change_time	TIMESTAMP	DEFAULT SYSTIMESTAMP
description	VARCHAR2(200)	-

2.2 Relationships and Cardinality

Relationship	Cardinality	How it is implemented
PATIENT has VISIT	1 : N	patient_id in VISIT (FK)
DOCTOR handles VISIT	1 : N	doctor_id in VISIT (FK)
BED is used in VISIT	1 : N (optional)	bed_id in VISIT (FK, nullable)
VISIT includes MEDICINE	M : N	PRESCRIPTION table with visit_id and medicine_id (FKs)
VISIT has BILL	1 : 1	visit_id in BILL (FK with UNIQUE)
AUDIT_LOG records changes	No FK	Filled by triggers on VISIT, PATIENT and BILL

2.3 Functional Requirements

Each functional requirement is mapped to the database component that will implement it. This shows how SQL and PL/SQL work together in the application.

No.	Functional requirement	Implemented using
FR1	Register a new patient and update patient details	INSERT / UPDATE on PATIENT; UNIQUE on phone
FR2	Search a patient and show full visit history	SELECT with JOIN (PATIENT, VISIT, DOCTOR)
FR3	Add and manage doctors with department, shift and fee	INSERT / UPDATE / DELETE on DOCTOR
FR4	Book an appointment; reject if the doctor already has one at that time	Procedure sp_book_appointment
FR5	Cancel an appointment	UPDATE on VISIT (visit_status)

No.	Functional requirement	Implemented using
FR6	Record an emergency visit with triage level	Procedure sp_register_emergency
FR7	List waiting emergency patients by triage level, then arrival time	Cursor with ORDER BY triage_level, visit_datetime
FR8	Mark doctor Busy / Available during treatment	Trigger on VISIT
FR9	Add medicines to a visit (prescription)	INSERT on PRESCRIPTION
FR10	Reduce medicine stock when prescribed; block if stock is zero	Trigger trg_reduce_stock on PRESCRIPTION
FR11	List medicines with low stock	SELECT with WHERE on stock_quantity
FR12	Show free beds of a ward type	SELECT on BED (bed_status = 'Free')
FR13	Allot a bed only if Free and mark it Occupied	Procedure sp_allot_bed
FR14	Discharge patient, release bed (Cleaning), record discharge time	Procedure sp_discharge_patient
FR15	Calculate number of days stayed	Function fn_days_stayed
FR16	Calculate medicine cost of a visit	Function fn_medicine_cost (JOIN PRESCRIPTION, MEDICINE)
FR17	Generate the bill (consultation + medicines + bed charges)	Procedure sp_generate_bill
FR18	Record payment and list pending bills	UPDATE / SELECT on BILL
FR19	Log every insert, update and delete automatically	Trigger trg_audit_visit / trg_audit_bill into AUDIT_LOG
FR20	Reports: cases per doctor, bed occupancy, revenue, triage-wise count	SELECT with GROUP BY, HAVING and subqueries

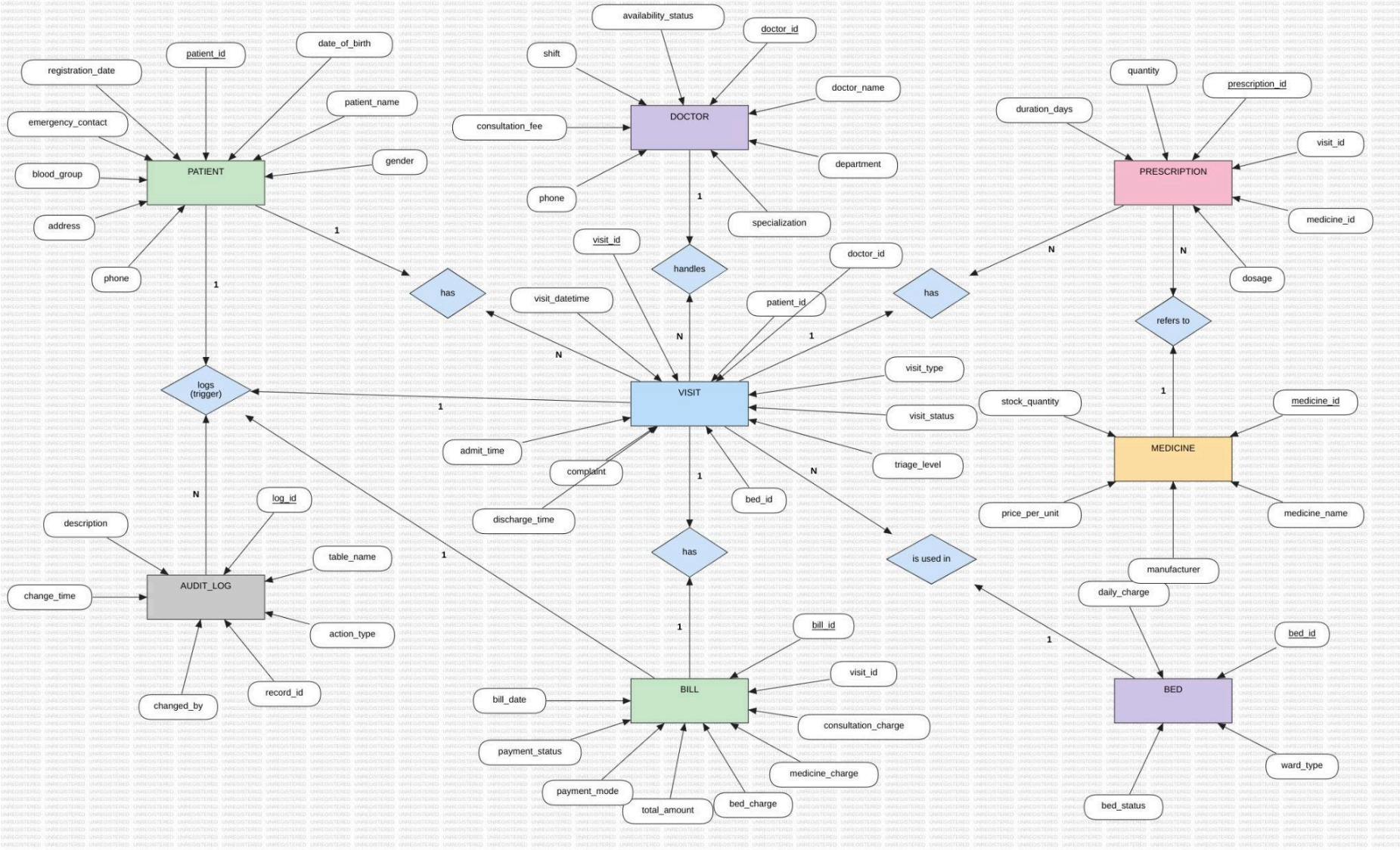
2.4 Planned Database Components (as per project guidelines)

Component	Minimum required	Planned in this project
Tables	6 to 8	8 tables: PATIENT, DOCTOR, BED, VISIT, MEDICINE, PRESCRIPTION, BILL, AUDIT_LOG
Constraints	NOT NULL, UNIQUE, CHECK, DEFAULT	Used across all tables (see section 2.1)
Records	At least 10 per table	10 or more sample records in every table
SQL queries	At least 15	Joins, GROUP BY, HAVING, subqueries, aggregate functions, ORDER BY
Procedures	At least 2	sp_book_appointment, sp_register_emergency, sp_allot_bed, sp_discharge_patient, sp_generate_bill
Functions	At least 2	fn_days_stayed, fn_medicine_cost
Cursor	At least 1	Waiting emergency patients in order of priority
Triggers	At least 2	trg_reduce_stock, trg_audit_visit (and trg_audit_bill)
INSERT / UPDATE / DELETE	Demonstration	On PATIENT, VISIT, BED and PRESCRIPTION
Connectivity	Java / Python etc.	Application connected to the database to run the operations interactively

2.5 Business Rules

- A patient's phone number must be unique.
- Triage level must be between 1 and 4 and is used only for emergency visits.
- A doctor cannot have two appointments at the same date and time.
- A bed can be allotted only if its status is Free.
- Discharge time cannot be before admit time.
- A medicine cannot be prescribed if its stock is zero.
- Fee, price and charges must be greater than zero.
- Audit log entries cannot be edited or deleted by normal users.

3. ER Diagram


availability_status
doctor_id
shift
doctor_name
consultation_fee
DOCTOR
department
phone
specialization
1
patient_id
date_of_birth
registration_date
patient_name
emergency_contact
gender
blood_group
PATIENT
address
phone
1
1
has
visit_id
handles
doctor_id
visit_datetime
patient_id
N
1
N
VISIT
logs
(trigger)
1
visit_type
visit_status
admit_time
triage_level
complaint
bed_id
discharge_time
N
1
N
log_id
description
table_name
change_time
AUDIT_LOG
action_type
changed_by
record_id
1
has
is used in
1
bill_id
visit_id
bill_date
BILL
consultation_charge
payment_status
medicine_charge
payment_mode
total_amount
bed_charge
quantity
prescription_id
duration_days
visit_id
PRESCRIPTION
medicine_id
N
N
has
dosage
refers to
1
stock_quantity
medicine_id
MEDICINE
price_per_unit
medicine_name
manufacturer
daily_charge
1
bed_id
BED
ward_type
bed_status

4. Relational Schema

The 8 entities of the ER diagram are mapped to 8 relations. Each strong entity becomes a relation with its identifier as the primary key. The 1 : N relationships are implemented by placing the primary key of the “one” side as a foreign key in the “many” side. The M : N relationship between VISIT and MEDICINE is resolved by the PRESCRIPTION relation, and the 1 : 1 relationship between VISIT and BILL is implemented with a UNIQUE foreign key.

4.1 Relational Schema

In the notation below, underlined attributes are primary keys and attributes marked with * (italic) are foreign keys.

Relation	Attributes
PATIENT	(patient_id, patient_name, gender, date_of_birth, phone, address, blood_group, emergency_contact, registration_date)
DOCTOR	(doctor_id, doctor_name, department, specialization, phone, consultation_fee, shift, availability_status)
BED	(bed_id, ward_type, bed_status, daily_charge)
VISIT	(visit_id, *patient_id* *, *doctor_id* *, *bed_id* *, visit_type, visit_datetime, complaint, triage_level, visit_status, admit_time, discharge_time)
MEDICINE	(medicine_id, medicine_name, manufacturer, price_per_unit, stock_quantity)
PRESCRIPTION	(prescription_id, *visit_id* *, *medicine_id* *, dosage, duration_days, quantity)
BILL	(bill_id, *visit_id* *, consultation_charge, medicine_charge, bed_charge, total_amount, payment_mode, payment_status, bill_date)
AUDIT_LOG	(log_id, table_name, action_type, record_id, changed_by, change_time, description)

4.2 Foreign Key Constraints

Foreign key	References (primary key)	Cardinality	Constraint
VISIT.patient_id	PATIENT.patient_id	N : 1	NOT NULL
VISIT.doctor_id	DOCTOR.doctor_id	N : 1	NOT NULL
VISIT.bed_id	BED.bed_id	N : 1 (optional)	NULL allowed (only for admitted patients)
PRESCRIPTION.visit_id	VISIT.visit_id	N : 1	NOT NULL
PRESCRIPTION.medicine_id	MEDICINE.medicine_id	N : 1	NOT NULL
BILL.visit_id	VISIT.visit_id	1 : 1	UNIQUE, NOT NULL

4.3 Relational Schema Diagram

